



# Recuperação de contexto para IA generativa em materiais educacionais abertos

Corretude, eficiência e qualidade de três estratégias de ordenação e busca sobre o livro *Pense Python*

João Cosme Sena Sá · Eduardo Henrique do Lago Silva · Helena Carvalho Leal

Hernandison da Silva Bispo · Nadianne Maria dos Santos Galvão · Rafael Takeguma Goto

**Disciplina:** Projeto e Análise de Algoritmos  
(PROCC0083), turma T01, 2026.2

**Atividade 1 (AV1):** Módulo I, Fundamentos e Eficiência  
na Era dos Dados · Tema 8

**Professor:** Glauco de Figueiredo Carneiro

**Apresentação:** 24/09/2026 · PROCC/UFS, São  
Cristóvão, SE

# Problema e corpus

- **Entrada:** corpus C com N chunks, consulta q em português, k resultados
- **Saída:** lista R com até k chunks, ordenada por relevância não crescente, sem repetição
- **Relevância** tf-idf lexical:  $tf = 1 + \ln f$ ,  $idf = \ln((N+1)/(df+1)) + 1$
- **Ordem total:** maior escore primeiro; empate decidido pela posição no livro (source\_order)
- **Casos de borda:** consulta vazia, k = 0 ou nenhum termo casando devolvem lista vazia; k maior que os candidatos devolve todos

## Pense Python

- Tradução livre da 3ª ed. de *Think Python* (Downey; trad. Carlson)
- 21 notebooks, 817.578 bytes, commit fixado e hash conferido
- 2.737 chunks (256 tokens, sobreposição 32, corte por célula)
- 30 consultas escritas antes dos resultados: 10 fáceis, 10 médias, 10 difíceis
- Texto CC BY-NC-SA: não versionado, baixado por script

# Quatro configurações, uma só função de relevância

| Config. | Busca                                           | Ordenação               | Papel               |
|---------|-------------------------------------------------|-------------------------|---------------------|
| C1      | varredura linear                                | Insertion Sort (equipe) | baseline            |
| C2      | índice invertido + busca binária no vocabulário | Merge Sort (equipe)     | busca indexada      |
| C3      | varredura linear                                | Merge Sort (equipe)     | divisão e conquista |
| C4      | TfidfVectorizer + cosseno (scikit-learn)        | biblioteca              | referência externa  |

C1, C2 e C3 devem devolver exatamente a mesma lista: só o custo muda. C4 normaliza os vetores (norma L2) e por isso pode devolver outra lista; é analisada à parte e não substitui nenhuma exigência.

*Cinco algoritmos obrigatórios cobertos: busca linear · Insertion Sort · Merge Sort (também a divisão e conquista) · busca binária · comparação com biblioteca*

# C2: índice invertido com busca binária

1

## Construção

cada termo do corpus ganha a lista crescente das posições dos chunks em que aparece; o vocabulário é ordenado uma vez

2

## Busca binária

cada termo da consulta é localizado no vocabulário em  $\Theta(\log V)$  comparações

3

## União

as listas dos termos encontrados são unidas com dois ponteiros, como no merge

4

## Pontuação e Merge Sort

só os candidatos da união são pontuados, com a mesma função de C1 e C3, e ordenados

## Corretude da busca binária

Só vale porque o vocabulário está ordenado pela mesma chave que o laço compara. Invariante: se o termo existe, está em `vocab[lo..hi]` no início de cada iteração.

## Custo real da consulta

$O(m \log V + m \cdot S + P(m + n))$

$S$  = soma dos postings dos termos,  $P$  = candidatos,  $n$  = tokens do chunk

# Corretude do Merge Sort

**Teorema.** Para uma lista finita de candidatos com `source_order` únicos e a ordem total  $\leq$  (maior escore; empate por menor `source_order`), `merge_sort` termina e devolve uma permutação da entrada ordenada por  $\leq$ .

## Invariante de `_merge`

- A saída está ordenada por  $\leq$
- Contém exatamente os elementos já consumidos de *left* e *right*
- Todo elemento da saída  $\leq$  qualquer elemento ainda não consumido
- Inicialização, manutenção e término verificados

## Indução no tamanho

- Base:  $|A| \leq 1$  devolve a própria lista
- Passo: as metades, menores que  $n$ , são corretas por hipótese; o `merge` junta preservando a ordem
- A recursão termina:  $\lfloor n/2 \rfloor < n$

## Limites do argumento

- Garante a ordenação, não a relevância semântica
- Exige escores finitos e ids únicos
- Testes: entradas embaralhadas, empates,  $k$  maior que os candidatos; 131 testes passando

# Modelo RAM, casos e recorrências

| Algoritmo                                       | Melhor                  | Pior                    | Médio                   | Espaço extra |
|-------------------------------------------------|-------------------------|-------------------------|-------------------------|--------------|
| Pontuação linear (N chunks, m termos, L tokens) | $\Theta(N \cdot m + L)$ | $\Theta(N \cdot m + L)$ | $\Theta(N \cdot m + L)$ | $\Theta(P)$  |
| Insertion Sort (P candidatos)                   | $\Theta(P)$             | $\Theta(P^2)$           | $\Theta(P^2)$           | $\Theta(1)$  |
| Merge Sort (P candidatos)                       | $\Theta(P \log P)$      | $\Theta(P \log P)$      | $\Theta(P \log P)$      | $\Theta(P)$  |
| Busca binária (V termos)                        | $\Theta(1)$             | $\Theta(\log V)$        | $\Theta(\log V)$        | $\Theta(1)$  |

## Merge Sort

$$T(P) = 2 \cdot T(P/2) + \Theta(P) \Rightarrow \Theta(P \log P)$$

duas chamadas nas metades; o merge percorre as duas uma vez

## Busca binária

$$T(V) = T(V/2) + \Theta(1) \Rightarrow \Theta(\log V)$$

uma chamada sobre metade do vocabulário;  $\Theta(1)$  no meio e na comparação

**Parâmetros declarados:** N chunks, L tokens do corpus, m termos da consulta, V termos distintos, P candidatos com escore  $> 0$ , k resultados.

A pontuação recalcula os termos de cada chunk a cada consulta, por isso o termo L entra no custo: no corpus inteiro,  $L = 72.205$  supera  $N \cdot m = 27.370$ .

# Metodologia experimental

**3.600**

execuções registradas, uma  
linha por execução

**$4 \times 3 \times 30$**

configurações  $\times$  tamanhos de  
corpus  $\times$  consultas

**$k = 5$  e  $10$**

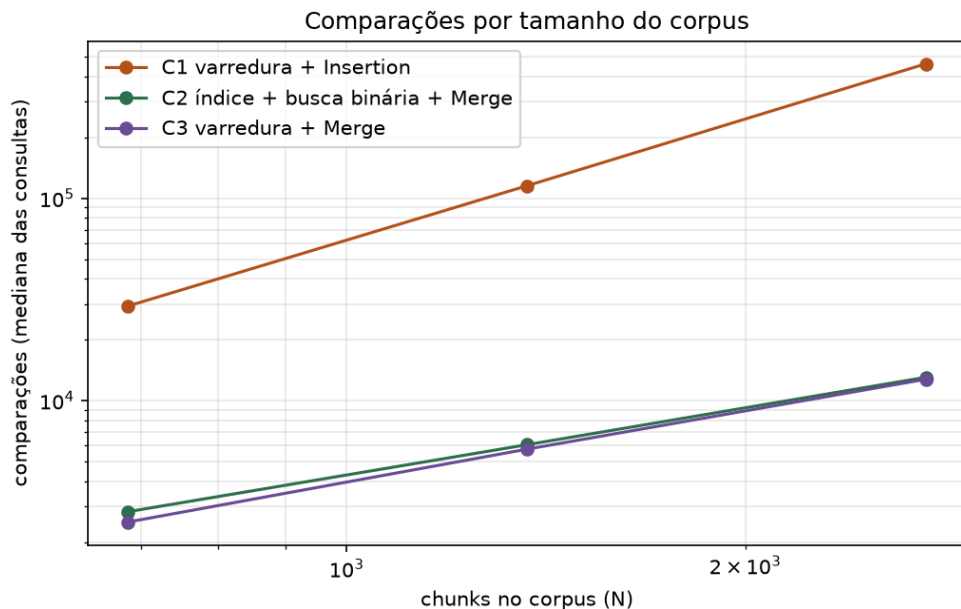
5 repetições após 2  
aquecimentos descartados

**0**

divergências entre C1, C2 e  
C3 em 180 combinações

- Tamanhos: 25 %, 50 % e 100 % dos chunks (684, 1.368 e 2.737), sorteados por capítulo com semente 2026
- Ordem das configurações sorteada em cada bloco; mediana e intervalo interquartil por célula
- Métricas: tempo de ingestão, índice, busca, ordenação, consulta e total; memória de pico (tracemalloc); comparações e trocas por contadores internos; Precision@k, Recall@k, nDCG@k e MRR contra o gabarito
- Ambiente registrado: Apple M5, macOS, Python 3.14.7, scikit-learn 1.6.1, commit e linha de comando
- Reprodução: `python scripts/reproduzir_tudo.py`, conferida em máquina limpa com resultados idênticos

# Comparações: quadrático contra $n \log n$



**$\times 4$  vs  $\times 2,2$**

comparações por dobra de candidatos: Insertion (C1)  
e Merge (C2, C3)

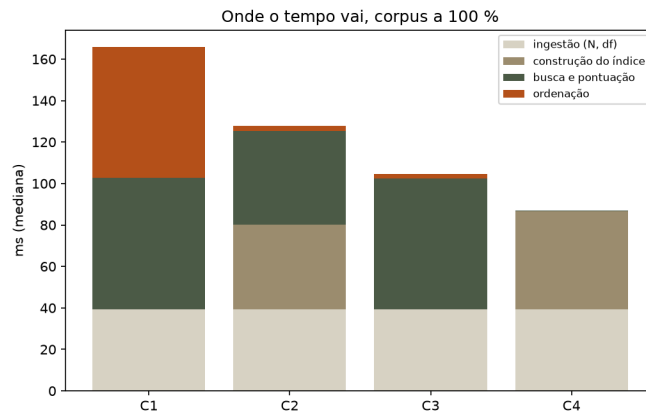
**$36\times$**

menos comparações no corpus inteiro para devolver  
a mesma lista

Insertion Sort fez 93 % de  $P^2/4$ ; Merge Sort, 85 % de  $P \log_2 P$  (medianas por consulta). A C2 soma poucas dezenas de comparações da busca binária.

# Tempo, memória e escalabilidade

| Configuração (corpus inteiro) | Total (ms) | Consulta (ms) | Comparações | Memória   |
|-------------------------------|------------|---------------|-------------|-----------|
| C1 varredura + Insertion      | 167,0      | 127,9         | 461.702     | 451 KiB   |
| C2 índice + binária + Merge   | 128,6      | 47,7          | 13.039      | 1.659 KiB |
| C3 varredura + Merge          | 105,7      | 65,8          | 12.746      | 452 KiB   |
| C4 scikit-learn               | 87,1       | 0,4           | n/a         | 2.581 KiB |



- C1 cresce 2,4× por dobra do corpus: a ordenação sobe de 13 % para 38 % do total
- C2 tem a consulta mais rápida da equipe (47,7 ms), mas paga 40,8 ms de índice; compensa a partir da 3ª consulta com o índice reaproveitado
- Preço da C2: 3,7× a memória da C3

# Qualidade da recuperação

| Configuração    | P@5   | P@10  | R@10  | nDCG@10 | MRR   |
|-----------------|-------|-------|-------|---------|-------|
| C1 = C2 = C3    | 0,087 | 0,077 | 0,158 | 0,141   | 0,248 |
| C4 scikit-learn | 0,247 | 0,183 | 0,387 | 0,366   | 0,551 |

**14/30**

consultas sem trecho  
relevante no top-10  
(equipe)

**2/30**

idem, na referência C4

- Referência: gabarito de 146 pares consulta–chunk, com justificativa, escolhidos antes de ver qualquer ranking
- C1, C2 e C3 têm a mesma qualidade porque devolvem a mesma lista: o algoritmo define o custo, não a qualidade
- A C4 acerta quase 3× mais: a norma L2 impede que trechos longos, como o Glossário (37 % do top-10 da equipe), dominem
- Nas consultas difíceis as duas empatam em nível baixo (0,10 vs 0,12): limite da busca lexical quando a pergunta usa palavras diferentes do livro

# Relação com IA generativa e RAG

## Prompt

os k chunks entram como bloco de contexto, cada um com a origem (notebook, seção) para citação

## k e tamanho do chunk

mais trechos e trechos maiores custam tokens e latência; a ordem importa (*lost in the middle*)

## Lexical vs. semântico

tf-idf casa palavras, não sinônimos: "laço" não acha "loop"; embeddings achariam

## Riscos

recuperação incompleta ou irrelevante em 14 das 30 consultas leva o LLM a responder sem base; contexto desatualizado é controlado pelo commit fixado

## Rastreabilidade

cada chunk carrega a proveniência: a resposta pode citar a fonte exata e o usuário confere

## Trade-offs

C4: 3× a precisão e consulta rápida, com 6× a memória da C3; C2: consulta rápida, com índice de 3,7× a memória

# Limitações, ameaças à validade e uso de IA

## Ameaças à validade

- A hipótese de esparsidade não se confirmou: palavras como "que" e "de" fazem 51 % dos chunks virar candidato, e o índice ganhou só 1,4× na consulta
- Três tamanhos mostram tendência, não provam ordem assintótica; o cronômetro mede o interpretador tanto quanto o algoritmo
- Os chunks seguem as células dos notebooks (mediana de 13 tokens), não janelas de 256
- Um livro introdutório é estudo de caso; julgamento de relevância binário e com uma avaliadora

## Uso crítico de IA generativa

- Registro por uso em docs/USO\_DE\_IA.md: ferramenta, finalidade, prompts, sugestões aceitas e rejeitadas
- Erros detectados: numeração de C2/C3 trocada; análise da C2 sem o termo da pontuação dos candidatos; empate tratado como simples estabilidade
- Nada entrou sem verificação: 131 testes, gabarito calculado à parte, contadores confrontados com  $P^2/4$  e  $P \log P$ , teste de mutação do Merge Sort

# O que aprendemos

## O algoritmo define o custo

C1, C2 e C3 devolvem a mesma lista com 36× de diferença nas comparações

## A função de relevância define a qualidade

a normalização L2 da C4 triplica a precisão sem mudar o algoritmo

## A hipótese precisa ser testada

o índice só compensa quando os termos são raros, e no português corrente não são

[github.com/joaocss/PAA\\_UFS\\_2026\\_2\\_SenaSa\\_Joao\\_Bispo\\_Hernandison\\_Galvao\\_Nadianne\\_Goto\\_Rafael\\_Leal\\_Helena\\_Silva\\_Eduardo](https://github.com/joaocss/PAA_UFS_2026_2_SenaSa_Joao_Bispo_Hernandison_Galvao_Nadianne_Goto_Rafael_Leal_Helena_Silva_Eduardo)

Reprodução completa: `python scripts/reproduzir_tudo.py`

# Referências

1. CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. *Introduction to Algorithms*. 4. ed. MIT Press, 2022.
2. KLEINBERG, J.; TARDOS, É. *Algorithm Design*. Pearson, 2006.
3. SKIENA, S. S. *The Algorithm Design Manual*. 3. ed. Springer, 2020.
4. SEDGEWICK, R.; WAYNE, K. *Algorithms*. 4. ed. Addison-Wesley, 2011.
5. MANNING, C. D.; RAGHAVAN, P.; SCHÜTZE, H. *Introduction to Information Retrieval*. Cambridge University Press, 2008.
6. LEWIS, P. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS*, 2020. arXiv:2005.11401.
7. ZHOU, Z. et al. A Survey on Efficient Inference for Large Language Models. arXiv:2404.14294, 2024.
8. GAN, A. et al. Retrieval Augmented Generation Evaluation in the Era of LLMs: A Comprehensive Survey. arXiv:2504.14891, 2025.
9. PEDREGOSA, F. et al. Scikit-learn: Machine Learning in Python. *JMLR*, v. 12, 2011.
10. FAISS. *Faiss documentation*. faiss.ai.
11. DOWNEY, A. B. *Think Python*. 3. ed. O'Reilly, 2024. Tradução livre *Pense Python*, por R. C. Carlson. rodrigocarlsongithub.io/PensePython3ed
12. CARNEIRO, G. F. *Projeto e Análise de Algoritmos: Atividade 1 (AV1) e aulas 02 a 05*. PROCC/UFS, 2026.2.